Лабораторна робота №5

Створення інтерфейсу мікросервісного веб-застосунку з використанням фреймворку ReactJS

Метою роботи є ознайомлення з фреймворком ReactJS та отримання навичок створення інтерфейсу з використанням ReactJS.

5.1 Теоретичні відомості

React – це популярна JavaScript-бібліотека, розроблена Facebook, яка використовується для створення інтерфейсів користувача, зокрема у форматі SPA (Single Page Applications). Вона ґрунтується на ідеї компонентної архітектури, де інтерфейс розбивається на окремі, повторно використовувані компоненти. Кожен компонент може мати власний стан і логіку, а при зміні стану React автоматично оновлює лише ту частину DOM, яка потребує змін. Такий підхід спрощує створення динамічного інтерфейсу та робить код зрозумілішим і модульним.

З архітектурної точки зору React виконує роль Presentation Layer – тобто шару подання в багаторівневих системах. У мікросервісному підході фронтенд (React) взаємодіє з окремими бекенд-сервісами через REST API. У нашому проєкті React замінив Django Templates і спілкується з кількома сервісами: DRF (для постів, коментарів, автентифікації), FastAPI (для рекомендацій). Таким чином, інтерфейс стає незалежним від реалізації бекенду, що підвищує гнучкість системи.

React використовує сучасні інструменти, зокрема Hooks – спеціальні функції (useState, useEffect, useContext), які дозволяють функціональним компонентам зберігати стан, виконувати побічні ефекти (наприклад, запити до API), підключатись до глобального контексту (наприклад, AuthContext). Наприклад, після логіну збережений токен у localStorage автоматично додається до кожного запиту. Також реалізована система оновлення токену (refresh) та його blacklist при виході з системи.

Замість Webpack або CRA, у проєкті використовується Vite – інструмент для запуску і збірки проєктів. Vite дозволяє миттєво запускати розробку, працює

через ES-модулі, забезпечує HMR (гаряче оновлення) і не потребує складної конфігурації. Він ідеально підходить для сучасних React-додатків.

Завдяки React Router реалізовано SPA-навігацію – сторінки змінюються без повного перезавантаження. Вбудована підтримка пагінації (через cursor у URL), умовний рендеринг залежно від авторизації, форми з FormData, реактивна логіка лайків і коментарів – усе це дозволяє студентам побачити, як React працює як інтегрований компонент у сучасній архітектурі ПЗ. React – це не просто бібліотека для рендеру UI, а потужний інструмент, який дозволяє будувати масштабовані, зручні й сучасні веб-застосунки.

5.2 Хід роботи

На минулих роботах ми розробили застосунок на основі мікросервісної архітектури, з використанням різних бекенд фреймворків, брокера RabbitMQ, а також Nginx в ролі API Gateway. Проте інтерфейс для нашого застосунку залишився нероздільним з монолітним Django застосунком. В цій роботі ми відключимо старий інтерфейс, та здійснимо перехід на React.

Архів з проектом React для роботи можна завантажити в системі Moodle за посиланням: https://eln.stu.cn.ua/mod/resource/view.php?id=315742 .

Щоб відключити старий інтерфейс, нам достатньо просто прибрати посилання на нього в файлі urls.py. Всі необхідні view для REST API перенесемо в файл api_views.py.

Лістинг 5.1 – Файл webapp/urls.py

```
from jango.urls import include, path
from rest_framework import routers
from knox import views as knox_views
from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView, TokenVerifyView, TokenBlacklistView
from . import api_views
router = routers.DefaultRouter()
router.register('authors', api_views.UserViewSet, basename='authors')
router.register('posts', api_views.BlogPostViewSet, basename='posts')
router.register('comments', api_views.PostCommentViewSet, basename='comments')
urlpatterns = [
```

```
        path("jwks", views.jwks_view, name="jwks"),
        path("api/", include(router.urls), name="api"),
                          path('api/token/',      TokenObtainPairView.as_view(),
name='token_obtain_pair'),
                       path('api/token/refresh/',     TokenRefreshView.as_view(),
name='token_refresh'),
                         path('api/token/verify/',     TokenVerifyView.as_view(),
name='token_verify'),
                    path('api/token/blacklist/',    TokenBlacklistView.as_view(),
name='token_blacklist'),
        path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
           path('api/docs/',  SpectacularSwaggerView.as_view(url_name='schema'),
name='swagger-ui'),
    ]
```

Для більшої зручності, змінимо логіку збереження зображень та вподобання дописів. Тепер зображення будемо зберігати в теці на диску (сервері), а не в базі даних. Це дозволить оптимізувати роботу з базою даних. Для вподобання допису додамо поле likes, яке відображатиме зв'язок ManyToMany з моделлю користувача, що дозволить визначати, який користувач вподобав допис.

Лістинг 5.2 – Файл models.py

```
    from django.db import models
    from django.contrib.auth import get_user_model
    User = get_user_model()
    class BlogPost(models.Model):
        title = models.CharField('Заголовок допису', max_length=56)
        text = models.TextField('Текст допису')
        like_count = models.IntegerField(default=0)
        author = models.ForeignKey(User, on_delete=models.CASCADE)
        last_modified = models.DateTimeField(auto_now=True)
        post_picture = models.BinaryField('Зображення допису', null=True)
             post_image  =  models.ImageField('Зображення  допису',  null=True,
blank=True, upload_to='blog_images/')
             likes  =  models.ManyToManyField(User,  related_name='liked_posts',
blank=True)
    class PostComment(models.Model):
        author = models.ForeignKey(User, on_delete=models.CASCADE)
        post = models.ForeignKey(BlogPost, on_delete=models.CASCADE)
        text = models.TextField(max_length=1024)
        last_modified = models.DateTimeField(auto_now=True)
```

В файлі settings.py встановимо пагінацію, папку та url для обробки та зберігання зображень, а також налаштування CORS.

Лістинг 5.3 – Файл settings.py

```
MIDDLEWARE = [
    <…>
    'corsheaders.middleware.CorsMiddleware',
    <…>
]
<…>
INSTALLED_APPS = [
    <…>
    'corsheaders',
    <…>
]
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
                                                    'DEFAULT_PAGINATION_CLASS':
'rest_framework.pagination.CursorPagination',
    'PAGE_SIZE': 10
}
CORS_ALLOWED_ORIGINS = [
    'http://127.0.0.1:5173',
    'http://localhost:5173'
]
USE_X_FORWARDED_HOST = True
CORS_ORIGIN_ALLOW_ALL = True
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Створимо файл міграції шляхом виконання python manage.py makemigrations, та модифікуємо його таким чином, щоб існуючі зображення були отримані з бази даних та збережені на диск (Лістинг 5.4).

Лістинг 5.4 – Файл migration<id>.py

```
from django.db import migrations, models
from django.core.files.base import ContentFile
from jango.conf import settings
```

```
import uuid
def move_binary_to_image(apps, schema_editor):
    BlogPost = apps.get_model('webapp', 'BlogPost')
    for obj in BlogPost.objects.all():
        if obj.post_picture:
            filename = f»{uuid.uuid4().hex}.jpg»
            obj.post_image.save(filename, ContentFile(obj.post_picture))
            obj.save()
class Migration(migrations.Migration):
    dependencies = [
        («webapp», «0002_alter_blogpost_last_modified_and_more»),
        migrations.swappable_dependency(settings.AUTH_USER_MODEL),
    ]
    operations = [
        migrations.AddField(
            model_name=»blogpost»,
            name=»post_image»,
            field=models.ImageField(
                blank=True,
                null=True,
                upload_to=»blog_images/»,
                verbose_name=»Зображення допису»,
            ),
        ),
        migrations.RunPython(move_binary_to_image),
        migrations.AddField(
            model_name="blogpost",
            name="likes",
            field=models.ManyToManyField(
                                blank=True,  related_name="liked_posts",
to=settings.AUTH_USER_MODEL
            ),
        )
]
```

Пізніше можна буде видалити старий атрибут `post_picture`. Тепер змінимо серіалізатор та view для BlogPost (Лістинг 5.5 – 5.6).

Лістинг 5.5 – Файл serializers.py

```
class BlogPostSerializer(serializers.ModelSerializer):
            comments  =  PostCommentSerializer(required=False,  many=True,
source="postcomment_set", read_only=True)
    author = UserSerializer(required=False, many=False, read_only=True)
    class Meta:
```

```
            model = BlogPost
          fields = ['id', 'author', 'title', 'text', 'comments', 'last_modified',
'post_image', 'likes']
            extra_kwargs = {'author': {'read_only': True}}
        def validate(self, attrs):
            author = self.context['request'].user
            attrs['author'] = author

            return super().validate(attrs)
```

Лістинг 5.6 – Файл api_views.py

```
    from rest_framework.pagination import CursorPagination
    from rest_framework import permissions, status, viewsets
    class IsAuthor(permissions.BasePermission):
        def has_object_permission(self, request, view, obj):
            return obj.author == request.user
    class BlogPostPagination(CursorPagination):
        ordering = '-last_modified'
    # <… інші view>
    class BlogPostViewSet(viewsets.ModelViewSet):
        # <… інші атрибути>
        pagination_class = BlogPostPagination
        # <… інші методи>
        def get_permissions(self):
            if self.action in ("list", "retrieve", "picture"):
                return []
            elif self.action == 'like':
                return [permissions.IsAuthenticated()]
            return [permissions.IsAuthenticated(), IsAuthor()]
        def get_queryset(self):
            author_id = self.request.query_params.get("author_id")
            queryset = BlogPost.objects
            if author_id:
                queryset = queryset.filter(author__id=author_id)
            return queryset.prefetch_related(
                Prefetch(
                                                                "postcomment_set",
queryset=PostComment.objects.select_related("author")
                ),
                'likes'
            ).all()
                                    @action(detail=True,        methods=['post'],
permission_classes=[permissions.IsAuthenticated])
        def like(self, request, pk=None):
```

```
        with transaction.atomic():
            post = self.get_object()
            user = request.user
            if post.likes.filter(id=user.id).exists():
                post.likes.remove(user)
                BlogPost.objects.filter(pk=post.pk)\
                .update(like_count=F('like_count') - 1)
                liked = False
            else:
                post.likes.add(user)
                BlogPost.objects.filter(pk=post.pk)\
                .update(like_count=F('like_count') + 1)
                liked = True
        post.refresh_from_db(fields=['like_count'])
        return Response({
            'liked': liked,
            'like_count': post.like_count,
        }, status=status.HTTP_200_OK)
```

Після змін в бекенді, приступимо до розробки фронтенд додатку. Завантажимо архів з системи Moodle, розархівуємо, та встановимо всі необхідні бібліотеки з використанням команди **npm install**. Зауважте, що версія node.js на вашому ПК має бути пізніша за 18, але найкраще використовувати найновішу доступну версію (в роботі – це 22.14.0). Після успішного встановлення всіх бібліотек, запустимо наш React проект, виконавши команду **npm run dev**. Локальний сервер розробки Vite повинен запуститись, а наш фронтенд має бути доступним за адресою http://localhost:5173. Зауважте, що для сервісу рекомендацій та Django ми додаємо до дозволених ресурсів як 127.0.0.1:5173 так і localhost:5173 .

Наш інтерфейс буде мати 5 основних сторінок: логін, реєстрація (треба буде реалізувати в якості самостійної роботи), перегляд дописів, перегляд окремого допису та додавання допису. Результуючий інтерфейс приведено на рисунках 5.1 – 5.5.

Основними React компонентами в нашому проекті є App.jsx. Тут ми визначаємо основні компоненти сторінки, основну секцію main та роутинг з використанням Route з react-route-dom.

Лістинг 5.7 – Файл App.jsx

```
import {useAuth} from './AuthContext';
import {Route, Routes, Link, useLocation} from 'react-router-dom';
import Posts from './Posts';
import PostDetail from './PostDetail';
import CreatePost from './CreatePost';
import Login from './Login';

function App() {
    const location = useLocation();
    const {isLoggedIn, logout, user} = useAuth();

    return (
        <>
            <header className="p-3 text-bg-dark">
             <div className="container d-flex justify-content-between align-
items-center">
                    <h3 className="text-white">
                      <Link to="/" state={{from: location}} className="text-
white text-decoration-none">Веб
                            блог</Link>
                    </h3>
                    <div>
                        {isLoggedIn ? (
                            <>
                            {user}  
                        <button className="btn btn-warning" onClick={logout}
>Вийти</button>
                            </>
                        ) : (
                            <>
                                <Link to="/login" state={{from: location}}
                                            className="btn btn-outline-light
me-2">Увійти</Link>
                                   <Link to="/register" className="btn btn-
warning">Зареєструватись</Link>
                            </>
                        )}
                    </div>
                </div>
            </header>

            <main className="container mt-4">
                <Routes>
```

```
                    <Route path="/" element={<Posts/>}/>
                    <Route path="/login" element={<Login/>}/>
                    <Route path="/posts/:id" element={<PostDetail/>}/>
                    <Route path="/create" element={<CreatePost/>}/>
                </Routes>
            </main>
        </>
    );
}

export default App;
```

Для авторизації, виходу та оновлення jwt слугує файл AuthContext.jsx. Тут ми створюємо context для всього нашого додатку, в якому будемо забезпечувати інформацію про поточного користувача, оновлювати його токен та виконувати вихід користувача. Для цього створюється функція-обгортка AuthProvider з різними методами, що повертає провайдер AuthContext.Provider, в який «обертаються» всі інші теги в застосунку.

У файлі main.jsx ми використовуємо AuthProvider, що надає усім дочірнім компонентам інформацію про поточного залогіненого користувача.

Лістинг 5.7 – Файл main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import {BrowserRouter} from 'react-router-dom';
import App from './App';
import {AuthProvider} from './AuthContext';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
    <React.StrictMode>
        <BrowserRouter>
            <AuthProvider>
                <App/>
            </AuthProvider>
        </BrowserRouter>
    </React.StrictMode>
);
```

Для кожної окремої сторінки ми маємо окремий файл: CreatePost.jsx, Login.jsx, PostDetails.jsx, Posts.jsx.

Файл auth.js містить корисні методи для збереження та отримання токенів з localStorage . Файл api.js містить методи для отримання даних з нашого REST API. Файл index.css використовується для глобального задання стилів, файл App.css використовується для стилів компоненту App. В нашому випадку, ми маємо однакові стилі в обох файлах.

У директорії public знаходиться файли зображень, зокрема стандартна favicon vite.svg, яку ми використовуємо в файлі App.jsx.

Точкою входу в сайт у випадку Vite слугує файл index.html – тут ми визначаємо основні бібліотеки, зокрема bootstrap, title нашого веб застосунку та шлях до файлу main.jsx.

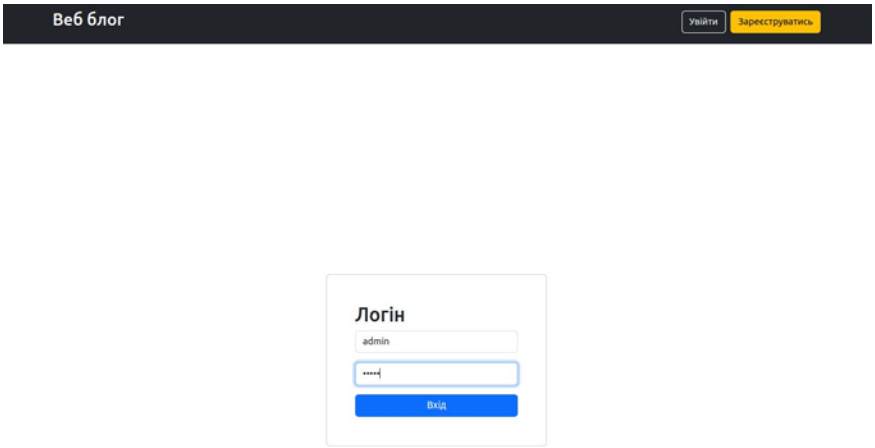

Рисунок 5.1 – Екран логіну

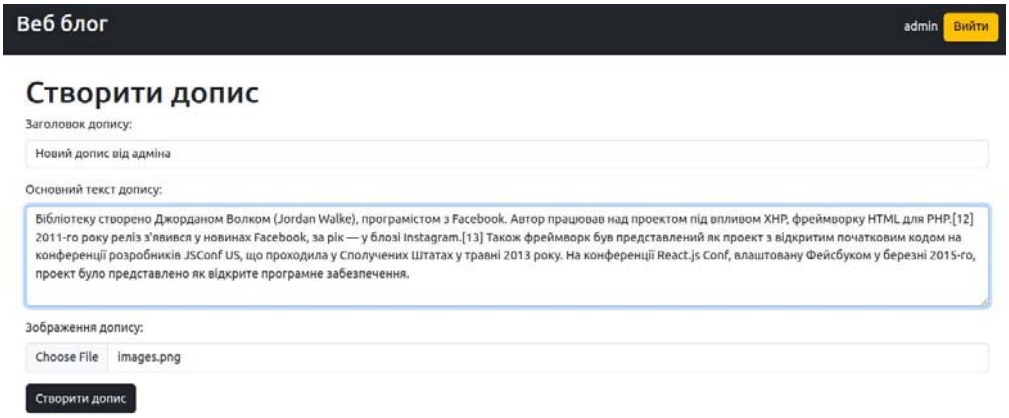

Рисунок 5.2 – Екран створення допису

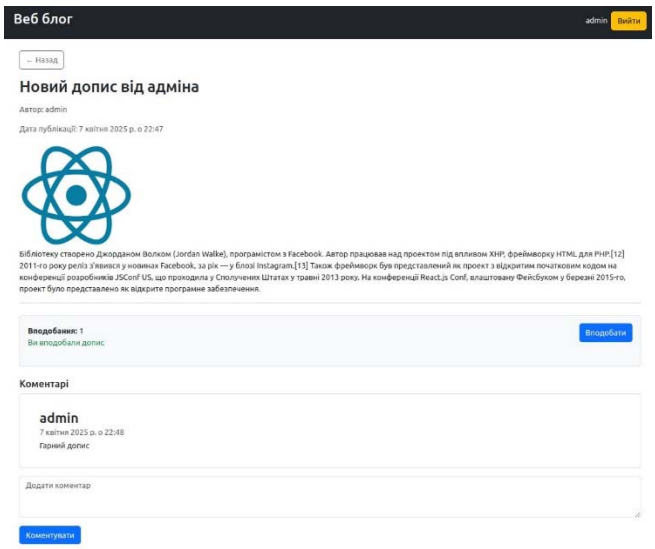

Рисунок 5.3 – Екран перегляду допису

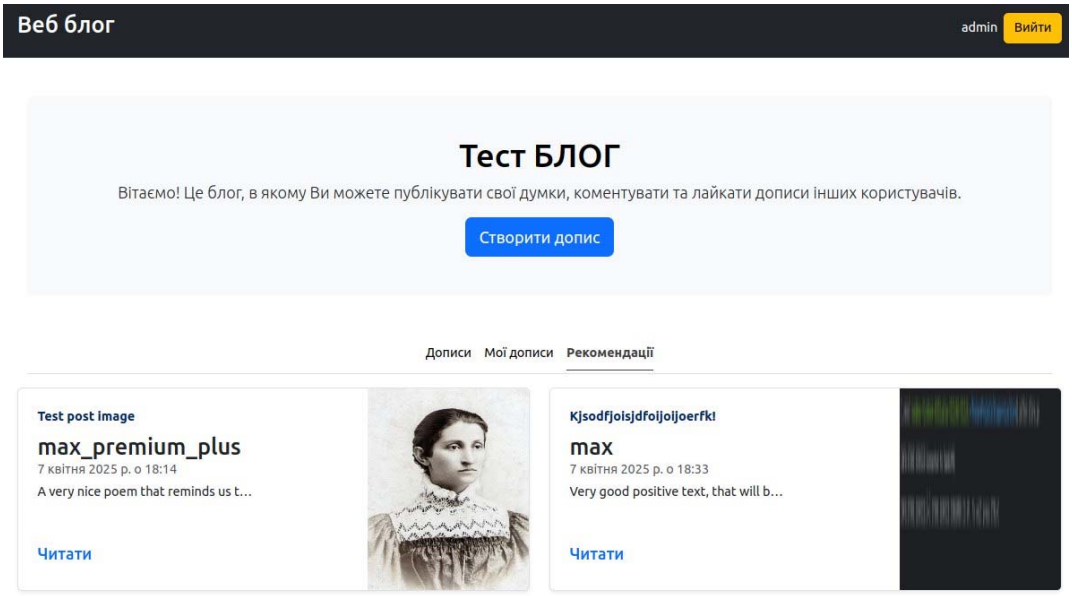

Рисунок 5.4 – Екран перегляду рекомендацій

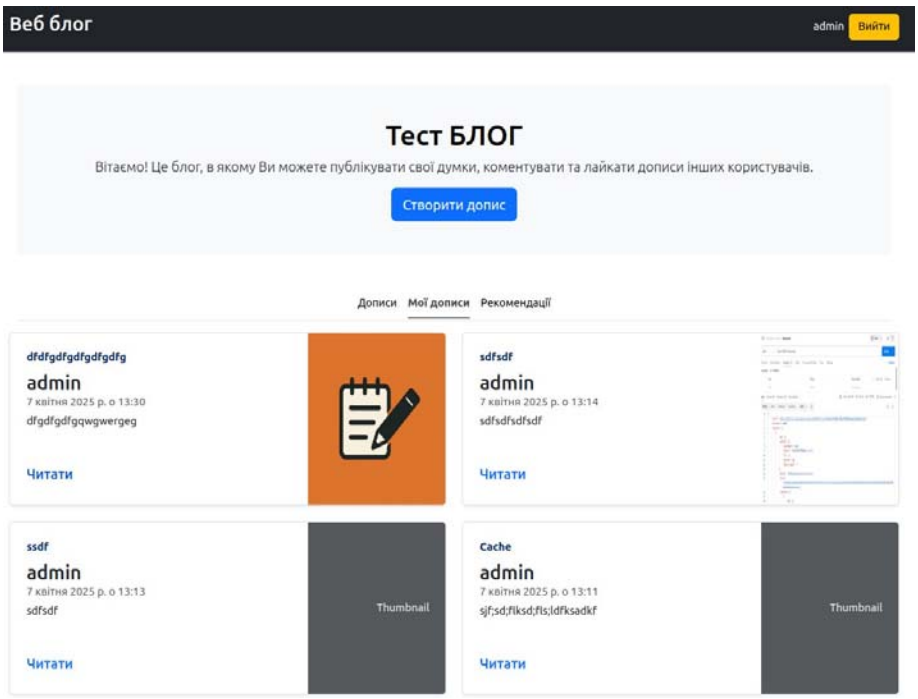

Рисунок 5.5 – Екран перегляду дописів автора

Запускати проект необхідно таким же чином, як і в лабораторній роботі №4: docker-compose -f local.yml up -d, потім запускати по черзі сервіси store, рекомендацій, сповіщень. Потім запускати DRF: python manage.py runserver . Важливо: для того, щоб Nginx міг коректно обробляти медіа файли дописів (зображення), треба в його конфігурацію додати media url та підвищити максимальний розмір файлів принаймні до 20 МБ (якщо будете завантажувати зображення до 20 МБ відповідно).

Лістинг 5.8 – Файл nginx.local.conf

```
events {}
http {
    server {
        listen 80;
        client_max_body_size 50M;
        location /health {
            add_header Content-Type text/plain;
            return 200 'healthy';
        }
        location / {
            proxy_pass http://127.0.0.1:8000/;
            proxy_set_header Host $host;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_no_cache 1;
            proxy_cache_bypass 1;
        }
        location /api/recommendations/ {
            proxy_pass http://127.0.0.1:8001/;
            proxy_set_header Host $host;
        }
        location /media/ {
            alias /blogproject/media/;
            autoindex off;
            try_files $uri =404;
        }
    }
}
```

5.3 Завдання на лабораторну роботу

1) Застосувати зміни до проекту відповідно до ходу роботи. Завантажити та запустити React проект. Переконатись, що він працює.

2) Реалізувати функцію реєстрації користувача з галочкою «підписатись на сповіщення». Користувач при реєстрації заповнює username та пароль (пароль 2 рази, паролі повинні співпадати). Пароль не коротше 8 символів. Форма реєстрації також має необов'язкове поле email, якщо користувач обрав «підписатись на сповіщення», то email обов'язковий до заповнення. Підписка на сповіщення працює відповідно до функціоналу, реалізованого в попередніх роботах.

3) Продемонструвати, що новостворений користувач може повністю взаємодіяти з сайтом.

5.4 Контрольні запитання

1. Опишіть основні особливості бібліотеки React.
2. Що таке хуки в React?
3. Що таке компонент у React?
4. Для чого використовують useState?
5. Для чого використовують useEffect?
6. Для чого використовують Context API?
7. За допомогою чого реалізується навігація в React проекті?
8. Опишіть реактивний підхід до створення інтерфейсів.